

Resolución del Práctico 1

Comandos, redirección, pipes, señales y llamadas al sistema

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es la resolución comentada del Práctico 1 de Sistemas Operativos (UNRC, 2026). Cada ejercicio se justifica con la teoría de los capítulos 1, 2 y 3 de las *Notas del curso* (Marcelo Arroyo) y del capítulo 1 de *Modern Operating Systems* (Tanenbaum & Bos). El objetivo es transparentar el modelo `fork()/exec()/wait()` junto con los mecanismos de redirección, pipes, señales y llamadas al sistema que provee la API POSIX. La meta es que un estudiante pueda leer este PDF de corrido y entender no solo *qué* comando se usó, sino *por qué*. Todo el código fue compilado con `gcc -Wall -Wextra` y verificado en distribuciones tipo UNIX (Linux y macOS); donde corresponda se aclaran las pocas diferencias entre ambos sistemas.

Índice

1. Introducción: por qué necesitamos un Sistema Operativo	2
2. Ejercicio 1 — Comandos del shell	2
3. Ejercicio 2 — Redirección, secuencia y filtrado	7
4. Ejercicio 3 — Pipes, sin archivos temporales	8
5. Ejercicio 4 — Shell scripts y permisos	9
6. Ejercicio 5 — Programa C con <code>exit status</code>	10
7. Ejercicio 6 — Versión propia de <code>system()</code>	11
8. Ejercicio 7 — Comunicación padre/hijo por pipe()	12
9. Ejercicio 8 — Comunicación por FIFO (named pipe)	13
10. Ejercicio 9 — Mini shell <code>minish</code>	14
11. Ejercicio 10 — Captura de SIGINT	15
12. Ejercicio 11 — Mínimo global con p hijos y un pipe	16
13. Ejercicio 12 — Lectura con <i>timeout</i> usando <i>alarm</i>	17
Cierre	19
Referencias	19

1. Introducción: por qué necesitamos un Sistema Operativo

Antes de entrar a los ejercicios conviene fijar el vocabulario que va a aparecer en todo el práctico, porque muchas respuestas dependen de distinguir bien qué está haciendo el **kernel** y qué está haciendo el **proceso de usuario**.

Un **sistema operativo (SO)** es la capa de software que abstrae el hardware, administra los recursos disponibles (CPU, memoria, dispositivos de E/S) y ofrece servicios a las aplicaciones mediante *llamadas al sistema*. Tanenbaum y Bos, en el capítulo 1 de *Modern Operating Systems*, lo presentan a la vez como:

- **Máquina extendida:** oculta la fealdad del hardware (controladores de disco, registros de DMA, interrupciones) detrás de abstracciones uniformes (archivos, procesos, sockets).
- **Administrador de recursos:** multiplexa CPU y memoria en *tiempo* y en *espacio* entre múltiples programas que corren simultáneamente.

Las Notas del curso enfatizan tres ideas transversales a este práctico:

Modo kernel y modo usuario

La CPU ejecuta el código del SO con privilegios totales (modo supervisor) y los procesos de usuario en modo restringido. El cambio se realiza vía *traps* (syscalls) o interrupciones. Una syscall no es una llamada a función común: es una instrucción especial que pide al hardware “deja lo que estás haciendo y entrá en modo privilegiado para atender este servicio”.

Proceso

Instancia de un programa en ejecución, con espacio de direcciones propio (código, datos globales, pila), descriptor en el kernel (PCB: *Process Control Block*) y archivos abiertos heredados de su padre.

Multitarea / multiprogramación

El SO mantiene varios procesos en memoria y multiplexa la CPU mediante *context switches*. En un SO de tiempo compartido el planificador es *preemptive*: puede sacarle la CPU a un proceso aunque éste no la libere voluntariamente.

Cómo se llama el kernel desde el código de usuario

Cuando un programa C invoca `read()`, no salta a una función del propio binario: ejecuta una instrucción de *trap* (en x86, `syscall`; en ARM, `svc`). La CPU cambia automáticamente a modo kernel, salva el estado del proceso y entrega el control a la tabla de syscalls del SO. Cuando el servicio termina, otra instrucción retorna a modo usuario en la línea siguiente al *trap*. Todo el modelo `fork/exec/pipe/signal` que recorre este práctico funciona así por dentro.

Sobre estas bases, el shell se entiende como un **intérprete REPL** (Read–Eval–Print Loop) que compone procesos mediante operadores de redirección (<, >, »), secuencia (;), condicionales (&&, ||), pipes (|) y ejecución concurrente (&). Cada uno de esos operadores tiene traducción directa en combinaciones de syscalls POSIX: `fork`, `exec`, `wait`, `pipe`, `dup2` y la familia `open/close/read/write`. Buena parte del práctico consiste, justamente, en hacer explícita esa correspondencia.

2. Ejercicio 1 — Comandos del shell

Los incisos de este ejercicio piden lo mismo: investigar el sistema con comandos básicos. Lo interesante no es la sintaxis (es trivial), sino *qué información del SO* está exponiendo cada comando. En cada inciso aclaramos esto último.

1.a Procesos en ejecución y PID del primer proceso

Planteo. Hay que listar los procesos vivos del sistema e identificar al primer proceso de usuario.

```
ps -e -o pid,ppid,state,comm | head -n 20
```

`ps -e` lista todos los procesos del sistema (no solo los del usuario actual). La opción `-o` elige las columnas: PID (id del proceso), PPID (id del padre), estado y nombre del comando. El proceso con **PID 1** es *siempre* el primer proceso lanzado por el kernel apenas concluye la inicialización del SO: el kernel lo crea “a mano” (no por `fork`) y a partir de ahí todos los demás procesos descienden de él.

Conclusión. En Linux moderno PID 1 suele ser `systemd` (o `init` en variantes clásicas); en macOS es `launchd`. Las Notas del curso lo describen como “el primer proceso de usuario pre-configurado”, y es la raíz del árbol de procesos: si `ps` muestra alguno con PPID 0, ese es el kernel mismo (no es un proceso de usuario, sino un *kernel thread*).

1.b Usuarios conectados y usuario actual

Planteo. Mostrar todos los usuarios con sesión activa y, en particular, quién soy yo.

```
who # todas las sesiones abiertas
who am i # solo la sesion asociada a esta terminal
whoami # UID efectivo del shell actual
```

Razonamiento. La diferencia entre los tres comandos ilustra la naturaleza multiusuario de UNIX:

- `who` consulta la base de sesiones del sistema (`utmp/wtmp` en Linux, `/var/run/utmpx` en macOS) y lista *todas* las sesiones que el SO considera activas.
- `who am i` filtra esa misma tabla por la terminal actual.
- `whoami` no mira la tabla de sesiones: imprime directamente el *UID efectivo* del proceso shell, que es el identificador de usuario con el que el kernel evalúa los permisos.

Conclusión. Las tres formas conviven porque responden preguntas distintas: “quién está logueado”, “con qué sesión vine yo” y “con qué identidad de usuario corre este proceso”.

1.c Directorio corriente

Planteo. Mostrar el directorio actual con dos formas distintas.

```
pwd
echo "$PWD"
```

Razonamiento. `pwd` consulta el *current working directory* del proceso shell, atributo que el kernel mantiene para cada proceso. La variable de entorno `$PWD`, en cambio, la mantiene el propio shell sincronizada cada vez que ejecuta `cd`. Ambos deberían coincidir, pero `$PWD` puede quedar desactualizado si otro proceso modifica el árbol de directorios por debajo.

Conclusión. Ambos comandos satisfacen el requisito del enunciado de mostrar el directorio actual con al menos dos formas distintas; el primero pregunta al kernel, el segundo lee una variable del propio shell.

1.d Directorio *home* del usuario

Planteo. Mostrar la ruta del directorio personal.

```
echo "$HOME"  
cd ~ && pwd
```

Razonamiento. HOME es una variable de entorno fijada en el momento del *login* a partir del séptimo campo del registro de `/etc/passwd` (o de la base de directorios equivalente en macOS). El símbolo `~` es un atajo que el shell expande a `$HOME` *antes* de ejecutar el comando, así que ambas formas terminan refiriéndose al mismo lugar.

1.e Metadatos de archivos

Planteo. Listar archivos con sus metadatos asociados.

```
ls -la . # listado del directorio actual  
ls -la / # listado de la raíz  
stat README.md / # metadatos detallados
```

Razonamiento. `ls -la` muestra entradas ocultas (`-a`) en formato largo (`-l`): permisos, dueño, grupo, tamaño, fecha. `stat` agrega información de bajo nivel típica del SO: número de inodo, cantidad de *hard links*, timestamps de acceso/modificación/cambio y dispositivo contenedor.

Conclusión. En UNIX un archivo es *una secuencia de bytes identificada por un inodo*; la entrada de directorio asocia un nombre legible con ese inodo. `stat` es la herramienta correcta cuando queremos ver explícitamente el inodo, porque `ls` lo oculta por defecto (se muestra con `ls -li`).

1.f Listar el contenido de un archivo sin abrir un editor

Planteo. Volcar el contenido a la terminal con tres comandos distintos.

```
cat README.md  
more README.md  
less README.md
```

Razonamiento. `cat` vuelca el archivo entero a `stdout` mediante un bucle `read()/write()` en bloques: muy simple, sin paginación. `more` agrega paginación hacia adelante (presionando espacio). `less` es la versión moderna: paginación bidireccional, búsqueda con `/patrón` y navegación tipo `vi`.

Conclusión. Para archivos cortos `cat` alcanza; para archivos largos conviene `less`, que además no carga el archivo completo en memoria.

1.g Variables de ambiente

Planteo. Listar las variables de entorno del shell actual.

```
env | sort
```

Razonamiento. Las variables de ambiente forman parte del *contexto* del proceso: son pares clave/valor que el kernel copia de padre a hijo cuando se hace `fork` y que el programa recibe en el tercer parámetro de `main` (`char *envp[]`) o accede vía `getenv()`. Por convención se escriben en mayúsculas.

Las más relevantes.

- USER, HOME: identidad y directorio personal del usuario.
- PWD: directorio de trabajo, sincronizado por el shell.
- SHELL: ruta al ejecutable del shell de *login*.
- PATH: lista de directorios donde se buscan los comandos (ver 1.h).
- LANG / LC_*: locale (idioma, codificación, formatos numéricos).

1.h Directorios donde el shell busca comandos

Planteo. Mostrar la lista PATH de forma legible.

```
echo "$PATH" | tr ':' '\n'
```

Razonamiento. PATH es una lista de directorios separados por dos puntos. Cuando el shell recibe un comando *sin barras* (por ejemplo `ls`, no `./ls`), recorre esos directorios en orden y ejecuta el primer archivo regular con permiso de ejecución que coincida con el nombre. `tr ':' '\n'` reemplaza los dos puntos por saltos de línea para que la lista se lea de corrido.

Por qué . no está en PATH

Por seguridad, el directorio actual (`.`) no suele estar en PATH. Si lo estuviera, un atacante podría dejar un binario malicioso llamado, por ejemplo, `ls` en un directorio público; al hacer `cd` ahí y tipear `ls`, terminarías ejecutando el binario malicioso en vez del comando del sistema.

1.i Ubicación de un comando con which

Planteo. Determinar el ejecutable concreto que se ejecutaría al tipear un nombre.

```
which cat
```

Razonamiento y conclusión. `which` reporta la *ruta absoluta* del primer ejecutable que el shell encontrará para ese nombre según el PATH vigente. Es útil para verificar qué versión de un comando se está ejecutando cuando hay varias instaladas (típico con `python`, `node`, `gcc`).

1.j Crear un archivo y agregarle al menos tres líneas

Planteo. Construir un archivo con varios comandos encadenados.

```
touch archivo.txt
echo "Primera línea" > archivo.txt
date "+%Y-%m-%d %H:%M:%S" >> archivo.txt
echo "Tercera línea" >> archivo.txt
cat archivo.txt
```

Razonamiento. `touch` crea el archivo si no existe (o actualiza su `mtime` si ya existe). El operador `>` **trunca y reescribe** el archivo destino, mientras que `>>` lo abre en modo *append*. La diferencia es crítica: si usáramos `>` en la tercera línea, perderíamos el contenido anterior.

Qué hace el shell por dentro. Antes de invocar `execve()` sobre el comando, el shell ejecuta:

1. Abre el archivo destino con `open(path, O_WRONLY|O_CREAT|O_TRUNC, 0666)` (o `O_APPEND` si era »).
2. Reasigna el descriptor 1 (`stdout`) al recién abierto con `dup2(fd, 1)`.
3. Cierra el descriptor sobrante con `close(fd)`.
4. Llama a `execve()` con los argumentos del comando.

El programa nunca se entera de la redirección: simplemente escribe en `stdout`, y resulta que `stdout` apunta al archivo.

1.k Crear un enlace simbólico

Planteo. Crear un *symlink* y observarlo.

```
ln -s archivo.txt archivo_link.txt
ls -li archivo.txt archivo_link.txt
```

Razonamiento. Un enlace simbólico (o *soft link*) es un archivo cuyo contenido es la *ruta* hacia el destino. Tiene su propio inodo y su propio tipo (1 al inicio de la línea de `ls -li`). `ls -li` muestra el número de inodo a la izquierda y la flecha `->` hacia el archivo real.

1.l Eliminar el symlink y compararlo con un hard link

Planteo. Borrar el enlace simbólico, crear un *hard link* y comparar ambos comportamientos.

```
rm archivo_link.txt
ln archivo.txt archivo_hard.txt
ls -li archivo.txt archivo_hard.txt
```

Razonamiento. Eliminar el symlink *no* afecta al original: solo se borra la entrada de directorio que contenía la ruta (o sea, el “cartelito”). El archivo real sigue donde estaba.

Un *hard link* es totalmente distinto: en lugar de guardar una ruta, crea otra entrada de directorio que **apunta al mismo inodo** que el archivo original. Es decir, no hay “original” y “copia”: hay un único inodo con dos nombres. El archivo desaparece del disco solo cuando el contador `st_nlink` (cantidad de nombres que apuntan al inodo) llega a cero *y* no hay procesos con el archivo abierto.

Característica	Symbolic link	Hard link
Tipo de referencia	Por ruta (texto)	Por inodo
Inodo propio	Sí	No (comparte)
Sobrevive al <i>rename</i> del original	Solo si la ruta sigue siendo válida	Sí
Sobrevive al borrado del original	No (queda <i>roto</i>)	Sí
Puede cruzar <i>filesystems</i>	Sí	No
Permitido sobre directorios	Sí	No (por defecto)

Cuadro 1: Comparación entre *symbolic links* y *hard links*.

Conclusión. El symlink es flexible (cruza filesystems, sirve para directorios) pero frágil (se rompe si el original se mueve o se borra). El hard link es robusto pero está limitado al mismo filesystem y, salvo excepciones reservadas al kernel, no se permite sobre directorios (para evitar ciclos).

1.m Programa C que muestra sus argumentos

Planteo. Escribir un programa que imprima `argc` y todos los `argv[i]`.

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[]) {
4     printf("argc = %d\n", argc);
5     for (int i = 0; i < argc; i++) {
6         printf("argv[%d] = %s\n", i, argv[i]);
7     }
8     return 0;
9 }
```

Compilación y ejecución:

```
gcc -Wall -Wextra -o hello hello.c
./hello arg1 arg2 arg3
```

Salida esperada:

```
argc = 4
argv[0] = ./hello
argv[1] = arg1
argv[2] = arg2
argv[3] = arg3
```

¿Por qué `./hello` y no simplemente `hello`? Cuando el usuario escribe un nombre *sin barras*, el shell asume que se trata de un comando externo y lo busca *únicamente* en los directorios listados en `PATH`. El directorio actual no suele estar en `PATH` (ver inciso 1.h, recuadro de seguridad), así que el shell no encontraría `hello`. El prefijo `./` indica una ruta relativa explícita: “ejecutá el archivo `hello` que está en mi directorio actual”. Como la ruta tiene una barra, el shell salta la búsqueda en `PATH` y va directo a `execve()`.

3. Ejercicio 2 — Redirección, secuencia y filtrado

Los archivos `correos1.txt` y `correos2.txt` contienen direcciones de correo (una por línea) con duplicados intencionales.

2.a Lista ordenada y sin duplicados con archivos temporales

Planteo. Combinar los dos archivos, ordenarlos y eliminar repetidos, pasando por archivos intermedios.

```
cat correos1.txt correos2.txt > /tmp/correos_todos.txt
sort /tmp/correos_todos.txt > /tmp/correos_ordenados.txt
uniq /tmp/correos_ordenados.txt > correos_ordenados_sin_duplicados.txt
```

Razonamiento paso a paso.

1. `cat` concatena ambos archivos y la redirección `>` envía el resultado al primer temporal.
2. `sort` ordena alfabéticamente las líneas: este paso es imprescindible porque `uniq` solo elimina duplicados *consecutivos*, no globales.

3. `uniq` colapsa las líneas idénticas adyacentes en una sola.

Los temporales se ubican en `/tmp` porque el SO purga ese directorio periódicamente y no contamina el directorio del trabajo.

2.b En una sola línea con operador secuencial ;

Planteo. Empaquetar los tres pasos en una única línea.

```
cat correos1.txt correos2.txt > /tmp/correos_todos.txt ; sort /tmp/correos_todos.txt > /tmp/correos_ordenados.txt ; uniq /tmp/correos_ordenados.txt > correos_ordenados_sin_duplicados.txt
```

Razonamiento. El operador `;` ejecuta los comandos en secuencia *independientemente* del éxito o fracaso del anterior: si `sort` fallara, `uniq` se ejecutaría igual (sobre un archivo vacío o incompleto, generando basura). Si quisiéramos abortar al primer error usaríamos `&&` en lugar de `;`. El enunciado, sin embargo, pide explícitamente el operador secuencial.

2.c Filtrar y contar direcciones de google.com

Planteo. Sobre el archivo final del inciso anterior, contar cuántas direcciones terminan en `@google.com`.

```
grep -E '@google\.com$' correos_ordenados_sin_duplicados.txt | wc -l
```

Razonamiento detallado.

- `grep -E` interpreta el patrón como expresión regular extendida.
- En el patrón `@google\.com$`:
 - `@google` pide coincidencia literal.
 - `\.` escapa el punto para que sea un punto literal, no el comodín “cualquier carácter”.
 - `com$` ancla el match al *fin de línea*, así no contamos casos como `usuario@google.com.ar`.
- El pipe (`|`) envía la salida de `grep` a `wc -l`, que cuenta las líneas resultantes.

Conclusión. Con los archivos de ejemplo provistos en la cátedra, el resultado esperado es 3.

4. Ejercicio 3 — Pipes, sin archivos temporales

Planteo. Resolver el mismo problema del ejercicio 2 pero sin escribir archivos intermedios en disco.

```
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
```

Qué cambió respecto del ejercicio 2. Los datos viajan por buffers en memoria del kernel (*pipe buffers*), nunca tocan el disco como archivo regular y los tres procesos (`cat`, `sort`, `uniq`) corren **en paralelo** comunicándose por esos pipes. Solo el resultado final cae al archivo de salida.

Cómo lo arma el shell por dentro. Las Notas del curso describen al *pipe* como un buffer acotado en el kernel que implementa el patrón productor/consumidor. Para `cmd1 | cmd2` el shell hace:

1. Crea el pipe con `pipe(p)`: el kernel devuelve dos descriptores, `p[0]` (lectura) y `p[1]` (escritura).

2. Llama a `fork()` dos veces (uno por cada lado del pipe).
3. En el proceso de la izquierda: `dup2(p[1], 1)` para que *stdout* apunte al extremo de escritura.
4. En el proceso de la derecha: `dup2(p[0], 0)` para que *stdin* apunte al extremo de lectura.
5. **Cierra los descriptores sobrantes en cada proceso.** Si quedaran abiertos, el lector nunca vería EOF (la condición de fin de archivo se dispara cuando el contador de escritores del pipe llega a cero).



Conclusión. La versión con pipes es más eficiente (no toca disco, paraleliza) y más limpia (no deja temporales). Es la forma idiomática en UNIX.

5. Ejercicio 4 — Shell scripts y permisos

4.1 Probar el script ejecutándolo con `sh`

Planteo. Empaquetar el pipeline del ejercicio 3 en un script y ejecutarlo invocando explícitamente al intérprete.

Archivo `procesar_correos_sin_interprete.sh` (sin *shebang*):

```
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
echo "Archivo generado: correos_ordenados_sin_duplicados.txt"
```

Invocación:

```
sh procesar_correos_sin_interprete.sh
```

Razonamiento. Al invocar `sh` explícitamente, el intérprete que ejecutará el script es el que el usuario eligió (`sh`); el archivo se pasa como argumento. Por eso **no necesita permiso de ejecución**: alcanza con poder leerlo, ya que `sh` lo abrirá con `open(..., O_RDONLY)` y lo interpretará línea por línea.

4.2 Agregar *shebang* y permisos de ejecución

Planteo. Hacer que el script se ejecute directamente, ya sin nombrar al intérprete.

```
#!/bin/sh
set -eu
cat correos1.txt correos2.txt | sort | uniq > correos_ordenados_sin_duplicados.txt
echo "Archivo generado: correos_ordenados_sin_duplicados.txt"
```

```
chmod +x procesar_correos.sh
./procesar_correos.sh
```

Razonamiento detallado.

- El *shebang* `#!/bin/sh` en la primera línea es leído por el kernel dentro de `execve()`: cuando el kernel detecta los dos primeros bytes `#!`, sustituye la ejecución del script por una ejecución del intérprete indicado, pasándole el script como argumento.

- `chmod +x` agrega el bit de ejecución; sin él, el kernel rechaza la llamada con `EACCES` (“permission denied”).
- `set -eu` es una buena práctica defensiva: `-e` aborta el script al primer comando que falle, `-u` marca como error el uso de variables no definidas. No es exigido por el enunciado pero evita errores silenciosos.

Conclusión. La gran diferencia conceptual con 4.1 es que aquí el kernel *no* sabe que es un script: simplemente intenta ejecutar el archivo, ve el shebang y delega al intérprete. Esto es lo que permite que existan scripts en cualquier lenguaje (`#!/usr/bin/env python3`, etc.) con la misma mecánica.

6. Ejercicio 5 — Programa C con `exit status`

Planteo. Escribir un programa que reciba un número como argumento y termine con ese valor como *exit status*, para luego combinarlo con los operadores `&&` y `||` del shell.

```

1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main(int argc, char *argv[]) {
6     char *endptr = NULL;
7     long value;
8
9     /* Se espera exactamente 1 argumento: el código de salida. */
10    if (argc != 2) {
11        fprintf(stderr, "Uso: %s <0-255>\n", argv[0]);
12        return 2;
13    }
14
15    /* strtol detecta errores de conversión numérica. */
16    errno = 0;
17    value = strtol(argv[1], &endptr, 10);
18
19    if (errno != 0 || endptr == argv[1] || *endptr != '\0') {
20        fprintf(stderr, "Error: el argumento debe ser un entero.\n");
21        return 2;
22    }
23
24    /* El shell maneja exit status en rango 0..255. */
25    if (value < 0 || value > 255) {
26        fprintf(stderr, "Error: el exit status válido está entre 0 y 255.\n");
27        return 2;
28    }
29
30    return (int)value;
31 }
```

Por qué validamos el rango `[0,255]`. Las Notas del curso describen `exit(code)` como la syscall que finaliza un proceso *limpiamente* y entrega el código al kernel. El padre puede recuperar ese código con `wait/waitpid`. Por convención 0 significa éxito y cualquier valor distinto, error. Sin embargo, el kernel solo expone los **8 bits inferiores** del exit code en la variable `$?` del shell, lo que justifica restringir el argumento al rango `[0, 255]`.

5.a Ejecutar y consultar el *exit status*

```
./myprog 7
echo $? # imprime 7
```

La variable especial `?` del shell contiene el exit status del último comando *en foreground*.

5.b Ejecutar otro comando solo si myprog tuvo éxito

```
./myprog 0 && echo "myprog termino con exito"
```

Razonamiento. El operador `&&` dispara el segundo comando *solo si* el primero retornó 0. Es la forma idiomática de encadenar pasos que dependen del éxito del anterior.

5.c Ejecutar otro comando solo si myprog tuvo error

```
./myprog 5 || echo "myprog termino con error"
```

Razonamiento. El operador `||` es el dual de `&&`: dispara el segundo comando solo si el primero retornó un valor *distinto de cero*.

Conclusión. Estos dos operadores permiten construir flujos de control completos a nivel shell, todo basado en un número entero “salida del proceso anterior” que el kernel mantiene en el PCB del proceso padre.

7. Ejercicio 6 — Versión propia de `system()`

Planteo. Reimplementar la función estándar `int system(const char *cmd)` usando directamente `fork`, `exec` y `wait`, para entender qué hay adentro de esa abstracción que tan a menudo se usa como caja negra.

```
1 #include <errno.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4
5 int mysystem(const char *command) {
6     pid_t pid;
7     int status;
8
9     /* Convencion POSIX: system(NULL) consulta si hay shell disponible. */
10    if (command == NULL) {
11        return access("/bin/sh", X_OK) == 0;
12    }
13
14    pid = fork();
15    if (pid < 0) return -1; /* fork fallo */
16
17    if (pid == 0) {
18        /* Hijo: lanza el comando a traves de /bin/sh -c "... " */
19        execl("/bin/sh", "sh", "-c", command, (char *)NULL);
20        _exit(127); /* exec fallo: convencion "comando no ejecutable" */
21    }
22
23    /* Padre: espera al hijo, tolerando interrupciones por senales. */
24    while (waitpid(pid, &status, 0) < 0) {
25        if (errno != EINTR) return -1;
26    }
27    return status;
28 }
```

Decisiones de diseño.

- **/bin/sh -c <cmd> en lugar de execvp.** Si hiciéramos directamente `execvp(command, ...)`, `command` tendría que ser un programa ejecutable; no podría contener pipes, redirecciones ni operadores como `&&`. Delegando en `/bin/sh -c` le permitimos al usuario pasar cualquier expresión válida del shell.
- **_exit(127) si exec falla.** La convención de `system()` es retornar 127 cuando el intérprete no pudo lanzarse. Usamos `_exit` (no `exit`) para evitar que las funciones registradas con `atexit` y los buffers de `stdio` corran dos veces (el hijo es una copia del padre, así que hereda esos buffers).
- **Reintento ante EINTR.** Una señal puede interrumpir `waitpid` antes de que el hijo termine realmente; en ese caso debemos reintentar, no abortar.

Cómo usar el status retornado. El `main` de prueba decodifica `status` con las macros `WIFEXITED` (¿el hijo terminó normalmente?) y `WEXITSTATUS` (¿cuál fue su código?). Esta mecánica de dos pasos — `fork` + `exec` — es exactamente la que el shell usa para implementar redirecciones: el padre, antes de `exec`, ajusta el ambiente del hijo (descriptores con `dup2`, variables de entorno) y solo entonces invoca el nuevo programa.

8. Ejercicio 7 — Comunicación padre/hijo por pipe()

Planteo. El padre debe pasarle un mensaje al hijo usando un *pipe* sin nombre. El hijo lee el mensaje y lo imprime por `stdout`.

Idea general. `pipe(p)` retorna dos descriptores: `p[0]` (lectura) y `p[1]` (escritura). Como `fork()` duplica los descriptores abiertos del padre, ambos procesos heredan ambos extremos del pipe. Cada uno debe cerrar el extremo que no va a usar; si no lo hace, el lector nunca verá EOF y se bloqueará indefinidamente.

```

1  int p[2];
2  pipe(p);
3  pid_t pid = fork();
4
5  if (pid == 0) {
6      /* Hijo: cierra extremo de escritura, lee del pipe e imprime. */
7      close(p[1]);
8      char buffer[256];
9      ssize_t n;
10     printf("Hijo recibio: ");
11     fflush(stdout);
12     while ((n = read(p[0], buffer, sizeof(buffer))) > 0)
13         write(STDOUT_FILENO, buffer, n);
14     close(p[0]);
15     _exit(0);
16 }
17
18 /* Padre: cierra extremo de lectura, escribe el mensaje y cierra. */
19 close(p[0]);
20 write(p[1], msg, strlen(msg));
21 close(p[1]); /* este close es el que dispara EOF en el hijo */
22 waitpid(pid, NULL, 0);

```

Compilación y ejecución:

```
gcc -Wall -Wextra -o pipe_padre_hijo pipe_padre_hijo.c
./pipe_padre_hijo "Hola desde el padre"
```

Salida esperada:

```
Hijo recibio: Hola desde el padre
```

El cierre del extremo de escritura es lo que dispara EOF

`read()` devuelve 0 (EOF) en un pipe *solo cuando no queda ningún proceso con el extremo de escritura abierto*. Por eso el padre tiene que cerrar `p[1]` después de escribir, y el hijo tiene que cerrar su copia heredada de `p[1]` antes de leer. Olvidar cualquiera de los dos cierres es la causa clásica de “el programa queda colgado” en este tipo de ejercicio.

Limitación importante. Un pipe sin nombre solo sirve entre procesos **emparentados**, porque los descriptores se heredan únicamente vía `fork()`. Si quisiéramos comunicar procesos arrancados de forma independiente, necesitamos un *named pipe*: eso es exactamente lo que resuelve el ejercicio 8.

9. Ejercicio 8 — Comunicación por FIFO (named pipe)

Planteo. Resolver el mismo problema del ejercicio 7, pero con un canal de comunicación que tenga un nombre en el sistema de archivos, de modo que procesos no emparentados también puedan usarlo.

Qué es un FIFO. Un FIFO (*named pipe*) es un pipe con un nombre en el sistema de archivos. Aparece en `ls -l` con la letra `p` al inicio de los permisos. Se crea con la syscall `mkfifo(path, mode)` (o el comando equivalente del shell) y se abre con la API estándar de archivos (`open`, `read/write`, `close`). Los datos *no se almacenan en disco*: solo el *nombre* vive en el filesystem; el buffer real está en el kernel, igual que en un pipe sin nombre.

```
1 #define FIFO_PATH "/tmp/practico1_fifo_pipe"
2
3 mkfifo(FIFO_PATH, 0600);
4
5 if (fork() == 0) {
6     /* Hijo lector. */
7     int fd = open(FIFO_PATH, O_RDONLY);
8     char buf[256];
9     ssize_t n;
10    printf("Hijo recibio: ");
11    fflush(stdout);
12    while ((n = read(fd, buf, sizeof(buf))) > 0)
13        write(STDOUT_FILENO, buf, n);
14    close(fd);
15    _exit(0);
16 }
17
18 /* Padre escritor. */
19 int fd = open(FIFO_PATH, O_WRONLY);
20 write(fd, msg, strlen(msg));
21 close(fd);
22 wait(NULL);
23 unlink(FIFO_PATH); /* el FIFO no se borra solo */
```

Sincronización implícita. `open(fifo, O_RDONLY)` se bloquea hasta que algún proceso abra el otro extremo con `O_WRONLY`, y viceversa. Esta propiedad es muy útil: dos procesos que no se conocen entre sí pueden coordinarse solo por la *ruta* del FIFO.

Por qué hace falta limpiar. A diferencia de un pipe sin nombre (que el kernel libera cuando se cierran sus extremos), el FIFO sobrevive como entrada del filesystem hasta que alguien lo borre explícitamente. Por eso al final del programa llamamos a `unlink()`.

10. Ejercicio 9 — Mini shell minish

Planteo. Implementar un mini intérprete de comandos en C que soporte los operadores básicos del shell. El objetivo didáctico es hacer explícita la correspondencia entre cada operador y las syscalls POSIX subyacentes.

Funcionalidad soportada. `minish` lee comandos en un bucle (REPL) hasta recibir `Ctrl-D` (EOF en `stdin`) y reconoce los siguientes operadores binarios:

Operador	Mecanismo POSIX
<code>cmd1 ; cmd2</code>	Ejecutar <code>cmd1</code> , luego <code>cmd2</code> .
<code>cmd1 && cmd2</code>	Ejecutar <code>cmd2</code> solo si <code>cmd1</code> retornó 0.
<code>cmd1 cmd2</code>	Ejecutar <code>cmd2</code> solo si <code>cmd1</code> retornó <code>!= 0</code> .
<code>cmd1 & cmd2</code>	<code>cmd1</code> en <i>background</i> ; <code>cmd2</code> en <i>foreground</i> .
<code>cmd1 cmd2</code>	<code>pipe(p)</code> + dos <code>fork/exec</code> + <code>dup2</code> .
<code>cmd < archivo</code>	<code>open(archivo, O_RDONLY) + dup2(fd, 0)</code> .
<code>cmd > archivo</code>	<code>open(..., O_WRONLY O_CREAT O_TRUNC) + dup2(fd, 1)</code> .
<code>cmd » archivo</code>	<code>open(..., O_WRONLY O_CREAT O_APPEND) + dup2(fd, 1)</code> .

Cuadro 2: Operadores reconocidos por `minish` y su mecanismo POSIX subyacente.

Estructura del programa.

1. **Bucle principal.** Llama a `getline()` para leer una línea. Si retorna `< 0` (EOF) el shell termina.
2. **Tokenización.** La línea se divide por espacios y tabulaciones; los tokens se almacenan en un arreglo de punteros al buffer original, modificado in situ con `\0`.
3. **Despacho recursivo (`dispatch`).** Busca operadores en orden de *menor a mayor prioridad* y descompone el comando en una terna (`left`, `op`, `right`), llamándose a sí mismo sobre cada lado. Esta recursión es la que respeta la precedencia natural de los operadores.
4. **Comando simple.** Cuando ya no hay operadores: hace `fork` + opcionalmente `dup2` (para redirecciones) + `execvp`. El padre llama a `waitpid`, salvo en *background*.
5. **Pipe.** Crea un pipe y hace dos `fork`; el padre cierra ambos extremos para que los hijos vean EOF correctamente cuando el productor termine.

Salida esperada en una sesión de prueba:

```
$ ./minish
minish$ echo hola ; echo mundo
hola
mundo
minish$ sleep 1 & echo "sigo en foreground"
[bg] pid=12345
sigo en foreground
minish$ true && echo "se ejecuto AND"
```

```

se ejecuto AND
minish$ false || echo "se ejecuto OR"
se ejecuto OR
minish$ echo hola | tr a-z A-Z
HOLA
minish$ ^D

```

Limitaciones intencionales. El parsing es deliberadamente simple: *minish* no soporta comillas, expansión de variables (*\$VAR*) ni *globbing* (**.txt*). El objetivo es ilustrar el modelo *fork/exec/wait/pipe/dup2*, no reimplementar *bash*. La implementación completa se encuentra en *resolucion-9/minish.c*.

11. Ejercicio 10 — Captura de SIGINT

Planteo. Escribir un programa que, ante un *Ctrl-C*, no termine de inmediato sino que pregunte al usuario si realmente quiere finalizar.

Marco teórico. Las Notas del curso describen las *señales* como un mecanismo asíncrono de notificación entre procesos o entre el kernel y un proceso. *SIGINT* (“interrupción”) es la señal que el driver de la terminal envía al proceso de *foreground* cuando el usuario presiona *Ctrl-C*. Sin un manejador instalado, el comportamiento por defecto del kernel es terminar el proceso.

Este ejercicio invierte ese comportamiento instalando un *signal handler* que pide confirmación.

```

1 #include <ctype.h>
2 #include <errno.h>
3 #include <signal.h>
4 #include <stdio.h>
5 #include <string.h>
6 #include <unistd.h>
7
8 static void sigint_handler(int sig) {
9     const char prompt[] =
10         "\nSIGINT recibida. Desea finalizar el proceso? [s/N]: ";
11     char answer[64];
12     ssize_t nread;
13     (void)sig;
14
15     /* write() es async-signal-safe; printf() no lo seria. */
16     write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);
17
18     /* read() puede ser interrumpida por otra senal: reintentamos. */
19     do {
20         nread = read(STDIN_FILENO, answer, sizeof(answer));
21     } while (nread < 0 && errno == EINTR);
22
23     for (ssize_t i = 0; i < nread; i++) {
24         unsigned char c = (unsigned char)answer[i];
25         if (!isspace(c)) {
26             if (c == 's' || c == 'S' || c == 'y' || c == 'Y')
27                 _exit(0);
28             _exit(-1);
29         }
30     }
31     _exit(-1);
32 }
33
34 int main(void) {
35     struct sigaction sa = {0};

```

```

36     sa.sa_handler = sigint_handler;
37     sigemptyset(&sa.sa_mask);
38     sa.sa_flags = SA_RESTART;
39     sigaction(SIGINT, &sa, NULL);
40
41     printf("Proceso en ejecucion. PID=%d\n", (int)getpid());
42     for (;;) pause(); /* duerme hasta que llegue una senal */
43 }

```

Por qué sigaction y no signal. La API `signal()` clásica tiene comportamiento histórico inconsistente entre sistemas (en algunos restablece el handler por defecto después de la primera entrega, en otros no). `sigaction` es la API moderna que da control fino sobre el comportamiento del handler (máscara, flags como `SA_RESTART`).

Funciones async-signal-safe. Un handler puede ejecutarse en *cualquier* momento, incluso en mitad de un `printf`. Si llamáramos a `printf` dentro del handler, podríamos engancharnos sobre un mutex interno de `stdio` y causar un *deadlock*. POSIX define un subconjunto reducido de funciones que son seguras en este contexto (`read`, `write`, `_exit`, `sigaction`, etc.); este código las usa exclusivamente.

Detalle sobre `_exit(-1)`. El kernel solo expone los 8 bits inferiores del exit status en `$_`, así que `_exit(-1)` se observa desde el shell como 255. El programa retorna *lógicamente* `-1`; la representación que ve el shell es solo un detalle de codificación.

12. Ejercicio 11 — Mínimo global con p hijos y un pipe

Planteo. Recibir por línea de comandos un parámetro `p` (cantidad de hijos) y luego un arreglo de enteros, y calcular el mínimo global en paralelo: cada hijo se ocupa de un trozo del arreglo y envía su mínimo local al padre por un único pipe compartido.

Esquema. El padre divide el arreglo en `p` bloques de `N/p` elementos (la asignación es directa: el hijo `i` trabaja sobre los índices `[i*chunk, (i+1)*chunk)`). Cada hijo calcula su mínimo local y lo envía al padre por el pipe; el padre lee `p` mensajes, va aplicando `min`, y reporta el resultado final tras esperar a todos los hijos con `waitpid`.

```

1  typedef struct {
2      int child_id;
3      int local_min;
4  } ChildResult;
5
6  /* ... validaciones de p y N (omitidas por brevedad) ... */
7  pipe(fd);
8
9  for (int i = 0; i < p; i++) {
10     pid_t pid = fork();
11     if (pid == 0) {
12         /* Hijo i: procesa su segmento. */
13         int start = i * chunk;
14         int end = start + chunk;
15         int local_min = arr[start];
16         for (int k = start + 1; k < end; k++)
17             if (arr[k] < local_min) local_min = arr[k];
18
19         ChildResult r = { i, local_min };
20         close(fd[0]); /* hijo no lee */
21         write_full(fd[1], &r, sizeof r); /* envia al padre */

```



```

22     close(fd[1]);
23     _exit(0);
24 }
25     children[i] = pid;
26 }
27
28 close(fd[1]); /* padre no escribe */
29
30 int global_min = INT_MAX;
31 for (int i = 0; i < p; i++) {
32     ChildResult r;
33     read_full(fd[0], &r, sizeof r);
34     if (r.local_min < global_min) global_min = r.local_min;
35 }
36 close(fd[0]);
37
38 /* Espera prolija a todos los hijos. */
39 for (int i = 0; i < p; i++) waitpid(children[i], NULL, 0);

```

¿Por qué un solo pipe alcanza para p escritores? POSIX garantiza que escrituras de hasta PIPE_BUF bytes son **atómicas** en un pipe: dos escritores nunca mezclan sus mensajes. Como `sizeof(ChildResult)` es muy pequeño (< 16 bytes), el padre puede leer mensajes completos en *cualquier orden* sin que se corrompan. No hace falta un pipe por hijo.

Sobre `write_full` / `read_full`. Los wrappers existen porque `read` y `write` pueden devolver menos bytes de los pedidos (lecturas/escrituras parciales). En este programa los mensajes son chicos y el pipe es local, pero como defensa general estos helpers acumulan los bytes en bucle hasta completar el tamaño solicitado, y reintentan ante `EINTR`.

Ejemplo de ejecución.

```

./minimo_global_pipe 3 8 4 7 10 -2 5
Hijo 0 -> minimo local = 4
Hijo 1 -> minimo local = 7
Hijo 2 -> minimo local = -2
Minimo global = -2

```

13. Ejercicio 12 — Lectura con *timeout* usando alarm

Planteo. Leer una línea de `stdin` pero solo durante un máximo de N segundos. Si vence el tiempo sin datos, salir con un código de error distintivo.

Marco teórico. Las Notas del curso muestran cómo implementar *timeouts* con la señal `SIGALRM`: el kernel envía esa señal al proceso cuando vence el tiempo programado con `alarm(N)`. El handler asociado puede marcar una bandera y, al retornar la syscall bloqueante (`read`), el programa decide si el retorno fue por *dato disponible* o por *interrupción*.

```

1 #include <errno.h>
2 #include <signal.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <unistd.h>
7
8 /* Bandera global segura para senales. */
9 static volatile sig_atomic_t timed_out = 0;
10

```

```

11  /* Handler de SIGALRM: solo marca el flag. */
12  static void alarm_handler(int sig) {
13      (void)sig;
14      timed_out = 1;
15  }
16
17  int main(int argc, char *argv[]) {
18      int N = atoi(argv[1]);
19      char buf[256];
20
21      struct sigaction sa = {0};
22      sa.sa_handler = alarm_handler;
23      sigemptyset(&sa.sa_mask);
24      sa.sa_flags = 0; /* importante: SIN SA_RESTART */
25      sigaction(SIGALRM, &sa, NULL);
26
27      alarm(N);
28      ssize_t n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
29      alarm(0); /* cancelar alarma si llego dato */
30
31      if (n > 0) {
32          buf[n] = '\0';
33          printf("Dato recibido: %s", buf);
34          return 0;
35      }
36      if (n == 0) {
37          puts("EOF recibido (stdin cerrado).");
38          return 0;
39      }
40      if (errno == EINTR && timed_out) {
41          printf("Timeout: no se recibieron datos en %d segundos.\n", N);
42          return 124;
43      }
44      perror("read");
45      return 1;
46  }

```

Por qué *sin* SA_RESTART. Si el handler se instalara con SA_RESTART, el kernel reanudaría automáticamente la syscall `read` después de entregar la señal y nunca veríamos EINTR. Es decir, el timeout sería invisible: `read` seguiría esperando datos como si nada. Quitando ese flag, `read` retorna -1 con `errno = EINTR`, y podemos detectar la condición consultando la bandera `timed_out`.

Por qué `volatile sig_atomic_t`. Un handler de señal puede modificar la bandera “en cualquier momento”. `volatile` impide al compilador asumir que la variable no cambia entre dos accesos (sin esto, podría optimizar la lectura y nunca enterarse del cambio). `sig_atomic_t` es el tipo más amplio que el estándar garantiza que se puede leer y escribir **atómicamente** desde un handler.

Por qué 124 como exit status. `coreutils` usa 124 por convención para indicar “terminación por timeout”, igual que el comando `timeout(1)`. No es un valor mágico, pero respetarlo facilita scripts que distinguen “vencido por tiempo” de otros errores genéricos.

Pruebas.

```

$ (sleep 1; echo "hola") | ./lee_con_timeout 5
Dato recibido: hola

$ (sleep 3; echo "tarde") | ./lee_con_timeout 1

```

```
Timeout: no se recibieron datos en 1 segundos.  
$ echo $?  
124
```

Cierre

A lo largo del práctico se ejercitó la API POSIX como manifestación concreta de los conceptos teóricos del SO:

- **Modelo de procesos:** `fork` / `exec` / `wait`, con su semántica de hijo como copia del padre y `exec` como reemplazo de la imagen de memoria.
- **Abstracción de archivos y descriptores:** `open` / `read` / `write` / `close` / `dup2`, base de redirecciones y pipes.
- **Mecanismos de comunicación entre procesos:** `pipe` (sin nombre, solo entre emparentados) y FIFO (con nombre en el filesystem).
- **Notificación asíncrona:** `signal` / `sigaction` / `alarm`, con sus restricciones (funciones `async-signal-safe`, `volatile sig_atomic_t`).

Los operadores del shell funcionan exactamente como combinaciones de esas syscalls. El mini shell del ejercicio 9 hace explícita esa correspondencia, y los ejercicios 10–12 muestran que aun el manejo de señales y timeouts, que suenan exóticos, son simplemente otra cara de la misma API. Los prácticos siguientes profundizan en planificación de CPU, gestión de memoria y sistemas de archivos, que aquí se asoman en su forma más cruda.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulos 1, 2 y 3. UNRC, 2026. <https://marceloarroyo.gitlab.io/cursos/SO/>
2. Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, capítulo 1, 5ª edición. Pearson, 2024.
3. POSIX.1-2017 / Single UNIX Specification. The Open Group, 2018.
4. Manuales de las syscalls (sección 2 de `man`): `man 2 fork`, `man 2 execve`, `man 2 pipe`, `man 2 sigaction`, `man 2 waitpid`, `man 3 system`.